Agentic-Wavelet Foundation Pipeline: Integrating Autonomous MLOps, Scattering Transforms, and In-Silico Neuroscience

1. Introduction and Architectural Vision

The evolution of computational neuroscience, affective computing, and neuro-technological diagnostics has reached a critical methodological inflection point. For decades, the foundational analytical techniques in electroencephalography (EEG) and related neuroimaging disciplines have relied on the Fast Fourier Transform (FFT) and classical matched filtering paradigms.[1] These traditional signal processing techniques operate under the stringent assumption of signal stationarity, decomposing time-series data into constituent global frequency components. However, the electrical activity of the human brain is profoundly dynamic, non-stationary, and stimulus-responsive.[1] Consequently, the reliance on global frequency representations often results in the smearing of transient, high-frequency temporal events—such as epileptic spikes, neural spindles, or localized affective arousal signatures—across the entire phase space, effectively burying critical diagnostic motifs within the environmental and biological noise floor.[1]

Simultaneously, the operational complexities associated with high-throughput biomedical data pipelines have escalated exponentially. The modern neuroinformatics landscape demands exhaustive data curation, precise artifact rejection, complex mathematical feature extraction, and the continuous training of high-capacity deep learning models. The manual execution of these pipelines has outpaced human engineering bandwidth, necessitating a paradigm shift toward autonomous machine learning operations (MLOps) driven by artificial intelligence.

To resolve these computational and operational bottlenecks, this report delineates the architecture for the Agentic-Wavelet Foundation Pipeline. This system represents a tripartite integration of three distinct technological frontiers. The first frontier is the Agentic Orchestration Layer, a terminal-native, autonomous software engineering and laboratory assistant. Constructed on the Bun JavaScript runtime and utilizing TypeScript with a React/Ink user interface, this layer draws direct architectural inspiration from production-grade CLI agents, specifically the mechanisms revealed within the 512,000-line codebase of Anthropic's Claude Code.[3] By leveraging a sophisticated QueryEngine and the Model Context Protocol (MCP), this layer autonomously executes scientific workflows, evaluates mathematical outputs, and triggers subsequent analytical stages without requiring human intervention.[3]

The second frontier is the Mathematical Preprocessing Engine, an enterprise-grade,

open-source Python library explicitly dedicated to the Wavelet Scattering Transform (WST). This analytical layer supplants the traditional FFT by employing cascading wavelet convolutions and modulus non-linearities.[1] The WST produces a deformation-invariant and translation-invariant representation of neural motifs, natively reducing dimensionality while meticulously preserving the high-frequency transient information critical for cognitive decoding.[1] This engine is designed to be fully decoupled from the primary agentic orchestrator, packaged and distributed via the Python Package Index (PyPI) to serve as an independent, highly scalable processing dependency.

The third and final frontier is the Foundation Modeling Layer. This layer introduces a multimodal, Transformer-based neural architecture inspired by Meta's Tribe V2 and Time-Frequency Consistency (TF-C) representation learning.[1] The objective of this layer is to project the low-dimensional WST scattering matrices into a highly contextualized latent space. By utilizing advanced neural tokenization methodologies, specifically the TFM-Tokenizer, the architecture maps discrete, localized neural motifs to broad cognitive states and functional cortical networks, enabling unprecedented capabilities in zero-shot anomaly detection and in-silico neurological simulation.[10] The subsequent sections of this report provide an exhaustive technical specification for designing, decoupling, and unifying these three frontiers into a coherent, high-performance research infrastructure.

2. The Agentic MLOps Architecture

The orchestration layer of the research pipeline must function as a persistent, stateful, and deterministic computational co-worker operating directly within the local terminal environment.[4] The architecture rejects the simplistic "chatbot wrapper" paradigm in favor of a complex, highly structured product stack.[13] The system is engineered using the Bun JavaScript runtime to capitalize on parallelized module resolution and rapid startup times, while relying on TypeScript to enforce strict, schema-driven type boundaries across all application contexts.[3]

2.1 Bounded Contexts and System Subsystems

To maintain architectural integrity and operational stability across a massive codebase, the Command Line Interface (CLI) application is strictly partitioned into logical bounded contexts, mirroring the architectural patterns observed in industry-leading AI agents.[3]

The Presentation and User Interface (UI) Context relies on a declarative rendering paradigm brought to the terminal environment. Instead of standard text output streaming, the application utilizes a React-reconciler mapped to ANSI escape codes via the Ink library.[5] This architectural decision enables the agent to render complex, interactive state components such as live progress bars during dataset splitting, interactive approval dialogues for destructive file modifications, and real-time metric visualization during Python script execution. The UI layer operates on a unidirectional data flow, consuming application state from a centralized store and dispatching user intents to the underlying execution engine while being defensively

wrapped in error boundaries to prevent terminal layout crashing.[5]

The QueryEngine serves as the central nervous system and primary orchestrator of the autonomous agent. Modeled after robust production implementations, this component manages the complex asynchronous generator loops that mediate interactions between the foundation Large Language Model (LLM) API, the local execution environment, and the persistent context window.[5] The core execution loop follows a strict, event-driven pipeline. It begins with context preparation, wherein the system gathers local state and retrieves persistent memory. To prevent context entropy—the tendency for an agent to hallucinate or lose operational focus as the prompt window expands during long-running tasks—the QueryEngine implements multi-tiered context compression. This includes local cleanup mechanisms, near-limit summarization routines, and emergency compression protocols that selectively re-inject only the most relevant operational facts.[3] Furthermore, the engine manages advanced streaming protocols, precise token estimation, and programmatic tool-calling iteration logic, enabling the agent to autonomously request an action, evaluate the result, and immediately request subsequent actions without stalling.[18]

For parallelizable MLOps workloads, such as executing hyperparameter sweeps across varying Wavelet Scattering configurations, the architecture employs a multi-agent orchestration pattern managed by a Coordinator module.[3] When the primary reasoning engine determines that a complex scientific goal can be decomposed, the Coordinator translates the objective into a directed acyclic graph of sub-tasks and spawns distinct, specialized sub-agents.[4] These sub-agents operate concurrently, communicating via a structured message bus and shared memory scratchpads. This swarm methodology allows the pipeline to simultaneously evaluate multiple mathematical preprocessing strategies, drastically reducing the temporal overhead of experimental neuro-informatics.[21]

Additionally, the architecture integrates a proactive daemon mode, operating as a background persistence layer. Inspired by systems like KAIROS, this background process conducts periodic memory consolidation, merging disparate scientific observations gathered during preprocessing runs, removing logical contradictions, and converting vague insights into verified, persistent operational facts.[3] This ensures that the agent actively maintains its understanding of the neuroscientific project state even when not actively prompted by the human researcher.[22]

2.2 Model Context Protocol (MCP) Integration

To ensure maximum extensibility and strictly decouple the TypeScript reasoning engine from the Python-based mathematical tooling, the architecture relies natively on the Model Context Protocol (MCP).[6] Traditional REST APIs require hardcoded endpoints, manual context stitching, and typed parameters that must be updated synchronously across clients and servers whenever an interface changes.[25] In contrast, MCP defines a standardized, bidirectional, and stateful communication protocol specifically engineered to connect LLM-powered applications

with external data sources and execution environments.[6]

The TS/Bun agent acts as the MCP Host and Client, while the independent Python scientific packages function as the MCP Servers.[6] Rather than hardcoding tool execution paths, the agent dynamically queries the MCP Servers at runtime to discover the available machine-readable capability surfaces, including resources, prompt templates, and executable tools.[25] This architecture supports advanced streaming semantics, enabling the Python MCP Server to push real-time progress notifications and partial mathematical results directly back into the agent's context loop, facilitating multi-step workflows that traditional stateless APIs cannot natively support.[23]

All data crossing the language barrier is governed by strict JSON-RPC messaging formats and validated using Zod schemas.[27] This rigorous validation ensures type safety at runtime, automating the creation of complex validation logic including type inference, cross-field refinements, and data transformations, thereby preventing the LLM from hallucinating invalid parameters or attempting malformed executions.[26]

2.3 QueryEngine Autonomous Evaluation Logic

A defining requirement for a next-generation laboratory assistant is the capacity to autonomously evaluate the output of a scientific process and deterministically decide the subsequent experimental step.[30] The QueryEngine achieves this by parsing structured JSON outputs emitted by the Python-based Wavelet Scattering MCP tools. When the agent invokes a data preprocessing script, the external tool executes the complex tensor operations and subsequently returns a highly structured output payload containing critical analytical metadata. This metadata includes quantitative measures such as the Signal-to-Noise Ratio (SNR), the count of anomalous or null values detected within the signal arrays, and the statistical variance of the resultant scattering coefficients.

The autonomous evaluation logic superimposed over the LLM's reasoning loop operates through a strict programmatic decision tree. Initially, the raw string output from the MCP tool is piped through a Zod schema parser to verify operational success and ensure the JSON payload is perfectly formed.[31] Subsequently, the system prompt injected into the QueryEngine applies explicit scientific threshold analysis. The agent is governed by predefined heuristic rules, such as directives to apply secondary denoising filters if the post-scattering SNR falls below a specified decibel threshold, or to proceed directly to dataset partitioning if the signal fidelity is adequate.

Based on the numeric values parsed and verified from the tool execution, the LLM utilizes its overarching system instructions to autonomously select the next appropriate MCP tool.[32] If the variance metadata indicates an anomaly—such as a massive, abrupt spike in the high-frequency detail coefficients characteristic of a disconnected EEG electrode—the agent recognizes the structural failure. It can then autonomously halt the WST pipeline, trigger a specialized artifact rejection tool to purge the corrupted epochs, and seamlessly resume the

preprocessing workflow without demanding human intervention.[30]

2.4 Custom MCP Tools for Scientific Orchestration

The functional capabilities of the agent are strictly delineated by its registered tools. Each tool is engineered as a discrete, bounded module that enforces precise permission models, input schemas, and execution contexts.[34] To successfully orchestrate the Agentic-Wavelet Foundation Pipeline, a specific suite of custom MCP tools must be implemented.

Tool Identifier	**Core Functionality**	**Execution Context**	**Required Permission Level**
ExecuteWSTTool	Triggers the open-source Python library to ingest raw EEG arrays and perform cascaded Wavelet Scattering Transforms.	External Python Subprocess via MCP RPC	Local File System Read/Write access; Subprocess execution.
DatasetSplitterTool	Partitions the transformed, low-dimensional scattering matrices into training, validation, and testing sets while enforcing strict subject-level grouping to prevent data leakage.	Native Bun/TypeScript File I/O	Local File System Read/Write access.
TribeInferenceTool	Submits the optimized WST feature matrices to the downstream biological foundation model for	External GPU Cluster / REST API Endpoint	Network and remote API access.

	TFM-tokenization and latent space anomaly detection.		
ArtifactRejectionTool	Scans raw EEG timeseries for non-physiological artifacts (e.g., electrode pops, extreme amplitude drifts) prior to transformation.	External Python Subprocess via MCP RPC	Local File System Read/Write access.

The implementation of the ExecuteWSTTool demands precise type definitions to satisfy the Model Context Protocol specifications.[36] By utilizing TypeScript and Zod, the system strictly defines the required input parameters, providing semantic descriptions that guide the LLM's automated reasoning.[28] The schema enforces constraints on critical wavelet hyperparameters, ensuring the agent cannot request invalid mathematical operations, such as an excessive scattering scale or an inappropriate number of wavelets per octave. When the LLM decides to execute the preprocessing step, it generates a JSON object that perfectly conforms to this schema, which the MCP client parses, validates, and forwards to the Python execution environment.[28]

3. The Open-Source Wavelet Engine: Mathematical Preprocessing

While the Bun-based agent excels at workflow orchestration, the intensive mathematical computation required for neuro-signal processing must be decoupled into an independent, high-performance Python package.[39] This open-source package is solely responsible for ingesting raw EEG data and applying the Wavelet Scattering Transform (WST).

The pervasive reliance on the Fast Fourier Transform (FFT) in biomedical engineering assumes signal stationarity, a condition fundamentally violated by the highly dynamic electrical activity of the human brain.[1] When an inherently non-stationary transient event—such as a millisecond-duration cognitive burst—is processed via FFT, its energy is distributed across the entire phase space.[1] The Short-Time Fourier Transform (STFT) attempts to mitigate this limitation by applying fixed temporal windows, but it remains strictly bound by the Heisenberg-Gabor uncertainty limit, forcing a persistent compromise between time and

frequency resolution that obscures transient detection.[1]

The Wavelet Scattering Transform circumvents these limitations by establishing a signal representation that is both translation-invariant and remarkably stable to small time-warping deformations.[1] By functioning as a complex-valued convolutional neural network where the filters are not learned but mathematically fixed as wavelets, the WST provides highly informative, low-variance features that preserve high-frequency information while algorithmically eliminating translation and rotation variability.[40]

3.1 The Mathematics of Wavelet Scattering

The WST constructs a robust representation through an iterative cascade of wavelet convolutions, modulus non-linearities, and low-pass filtering operations.[1] Let a continuous input signal $x(t)$ be convolved with a complex wavelet $\psi_\lambda(t)$ characterized by a center frequency λ. The transformation at a specified scale is defined by a low-pass scaling filter $\phi(t)$ with a spatial window of size 2^J.[1]

The zero-order scattering coefficient (S_0) represents the local average of the signal, analogous to the direct current (DC) component. It is computed simply by convolving the signal with the low-pass filter:

$$S_0 x(t) = x * \phi(t)$$

The first-order scattering coefficient (S_1) captures the signal's energy distribution across distinct frequency bands, functioning similarly to a traditional scalogram. It is generated by applying the wavelet transform, computing the complex modulus to discard phase variation and ensure translation invariance, and finally averaging the result:

$$S_1 x(t, \lambda_1) = |x * \psi_{\lambda_1}| * \phi(t)$$

The second-order scattering coefficient (S_2) represents the critical advancement for capturing complex transient neural motifs. It takes the modulus of the first layer—which effectively acts as a low-frequency envelope of the high-frequency activity—and convolves it with a secondary bank of wavelets. This cascaded operation captures intricate amplitude modulations and inter-scale temporal dependencies that linear transformations completely miss [1]:

$$S_2 x(t, \lambda_1, \lambda_2) = ||x * \psi_{\lambda_1}| * \psi_{\lambda_2}| * \phi(t)$$

The resulting feature matrix is dense, highly compressed, and theoretically robust. The scattering representation mathematically guarantees Lipschitz continuity to minor

deformations. If a deformed signal is defined as $x_\tau(t) = x(t - \tau(t))$ where the deformation $\tau(t)$ is small, the scattering transform guarantees that $||\Phi(x) - \Phi(x_\tau)|| \leq C||\nabla\tau||_\infty||x||$.[1] This crucial property ensures that minor biological variances in the precise timing or duration of EEG spikes do not drastically distort the resultant feature vector, vastly improving the signal-to-noise ratio prior to ingestion by deep neural networks.

3.2 Kymatio Integration and Hyperparameter Tuning

The Python package wraps Kymatio, the industry-standard software package for executing wavelet scattering transforms. Kymatio is highly optimized for integration with deep learning frameworks and supports aggressive GPU acceleration.[46] A significant architectural advantage of Kymatio is its depth-first traversal of the scattering transform tree, which contrasts with the breadth-first (layer-by-layer) approach utilized by legacy toolboxes such as ScatNet. This depth-first strategy limits memory footprint and is substantially better suited for execution on parallelized GPU hardware.[7]

The structural dimensionality of the output tensor depends heavily on two critical hyperparameters: the maximum scale J and the frequency resolution Q.[7] The parameter J specifies the maximum scale of the low-pass filters as a power of two, defining the temporal integration window.[7] For an EEG signal sampled at a standard clinical frequency, selecting an optimal J ensures that the largest filter is concentrated over a time interval conducive to capturing relevant neural micro-states.

The parameter Q is defined as a tuple (Q_1, Q_2) controlling the number of wavelets per octave in the first- and second-order filter banks.[45] For highly oscillatory biological signals such as EEG, which are often better localized in frequency than in time, it is highly beneficial to set a large value for Q_1 (typically between 8 and 16). The second layer resolution, Q_2, is usually set to 1 or 2 to effectively capture the broader amplitude modulation spectrum.[45]

For a one-dimensional input tensor of shape (B, T), where B represents the batch size and T denotes the number of discrete time samples, the Kymatio 1D scattering transform produces an output tensor of shape $(B, P, T/2^J)$.[7] Here, P indicates the total number of distinct scattering paths (coefficients). This operation drastically compresses the temporal dimension by a factor of 2^J while expanding the representational depth along the feature dimension P.[7]

3.3 Dimensionality Reduction for Transformer Tokenization

While the WST inherently downsamples the signal temporally, the proliferation of scattering paths P across the first and second orders can yield a prohibitively large feature dimension, posing challenges for the tokenization bottlenecks of downstream foundation models.[50] To optimize the scattering matrices for efficient ingestion, the open-source package implements a rigorous dimensionality reduction pipeline.

Following the extraction of the zero, first, and second-order scattering features, Principal Component Analysis (PCA) is applied across the scattering path dimension.[50] This algorithmic step reduces the high dimensionality of the scattering characteristics, projecting the complex tensors into a dense, linearly uncorrelated latent vector space.[50] By retaining only the principal components necessary to explain 95% of the variance, the framework drastically increases computational efficiency while preserving the subtle, discriminative edge-like patterns captured by the non-linear modulus operators.[42] The final output is an optimal, low-dimensional "neural motif" representation perfectly suited for discrete tokenization.

3.4 Enterprise-Grade PyPI Project Structure

To fulfill the strict open-source requirement, the WST preprocessing logic is entirely decoupled from the TypeScript agent and packaged for broad distribution via the Python Package Index (PyPI). The project architecture rigorously adheres to the modern standard src/ layout, governed by pyproject.toml.[39] This structure prevents common import resolution bugs, isolates the package code during testing, and aligns with the highest standards of the Scientific Python community.[52]

Directory / File Path	**Architectural Purpose**
pyproject.toml	Centralized build system requirements, PEP 621 metadata, and strict dependency declarations.
README.md	Comprehensive usage documentation detailing integration with MCP servers.
src/eeg_wst/__init__.py	Package initialization exposing the primary public API classes for the pipeline.

src/eeg_wst/core.py	Instantiation of Kymatio Scattering1D objects and the core cascaded transform logic.
src/eeg_wst/reduction.py	Implementation of the PCA dimensionality reduction algorithms post-scattering.
src/eeg_wst/io.py	Utilities for loading and parsing raw neuro-signals from standard formats (e.g., EDF, FIF).
tests/test_scattering.py	Pytest suite validating Lipschitz continuity and ensuring output tensor shape conformity.

The pyproject.toml explicitly defines external dependencies including numpy, scipy, and kymatio.[47] Through this decoupled design, the overarching Agentic CLI securely installs the PyPI package into an isolated virtual environment, invoking the heavy mathematical computations strictly through the MCP Server interface without polluting the orchestration runtime.

4. Foundation Integration (The Tribe V2 Layer)

The final architectural tier involves the integration of the biological foundation model. After the autonomous agent utilizes the ExecuteWSTTool to transmute raw EEG data into stable, low-dimensional scattering matrices, these mathematical arrays must be mapped into a semantic space representing functional cognitive dynamics. This advanced pipeline draws deep architectural and theoretical inspiration from Meta's TRIBE v2 model and Time-Frequency Consistency (TF-C) representation learning.[1]

4.1 The Tribe V2 Architecture Paradigm

TRIBE v2 constitutes a massive tri-modal foundation model engineered for in-silico neuroscience, capable of predicting human brain activity across 70,000 high-resolution fMRI voxels in response to diverse naturalistic stimuli.[1] The architecture processes multimodal inputs by extracting embeddings from three state-of-the-art pretrained AI models: V-JEPA for vision, Wav2Vec-BERT for audition, and Llama 3.2 for language.[1] These embeddings are integrated using a unified Transformer architecture, which maps the shared latent representations directly

onto the fsaverage5 cortical surface mesh.[1]

Crucially, to accommodate the vast biological variability across hundreds of subjects, TRIBE v2 employs a specialized "Subject Block." This component utilizes 16 learnable parameters within a subject-conditional linear layer to adapt the universal transformer outputs to individual neurological topologies.[1]

Our Agentic-Wavelet Foundation Pipeline essentially inverts the TRIBE v2 paradigm. Whereas TRIBE v2 maps external stimuli to resultant brain activity, our architecture maps preprocessed brain activity (the WST scattering matrices) to functional cognitive states or discrete anomaly classifications. The WST matrices serve as the highly refined input embeddings, analogous to the multimodal feature vectors used in TRIBE v2. Furthermore, the inclusion of a subject-conditional layer ensures the downstream Transformer can normalize individual electrophysiological baselines, a critical requirement for generalized Brain-Computer Interface (BCI) applications.

4.2 Tokenization of Neural Dynamics via TFM-Tokenizer

Feeding continuous mathematical matrices into a discrete Transformer architecture demands highly sophisticated tokenization strategies.[10] The pipeline integrates the core logic of the TFM-Tokenizer (Time-Frequency Motif Tokenizer), a framework specifically tailored for advanced EEG analysis.[10]

Conventional tokenization approaches typically rely on continuous temporal embeddings and rigid inter-channel dependencies, rendering them highly vulnerable to temporally unpredictable patterns and diverse oscillatory waveforms.[10] The TFM-Tokenizer mitigates this limitation by autonomously learning a discrete vocabulary of "neural motifs"—recurrent, transient structural patterns localized in both the time and frequency domains—from single-channel data.[10]

This tokenization is achieved via a dual-path architecture featuring a Localized Spectral Window Encoder.[11] The WST coefficients generated by the Python package intrinsically contain this rich time-frequency information. The spectral window encoder processes these matrices, aggregating information across the various scattering paths and fusing the frequency band data with raw temporal context.[11] The tokenizer is pretrained using a rigorous masked token prediction objective. By strategically masking specific temporal windows or frequency bands and forcing the model to reconstruct the missing data, the system ensures that the resulting discrete tokens are dense, class-discriminative representations of genuine neurophysiological events.[11] These discrete tokens then form the exact sequence utilized as input for the downstream foundation model.

4.3 Time-Frequency Consistency (TF-C) Integration

To guarantee that the downstream Transformer comprehensively understands the profound physical relationship between the precise timing of a neural spike and its corresponding

oscillatory frequency, the pipeline heavily integrates Time-Frequency Consistency (TF-C) contrastive learning.[1]

The TF-C framework posits a foundational principle: a time-based neighborhood of a specific EEG sample must be embedded within the latent space in close proximity to its corresponding frequency-based neighborhood.[1] To realize this consistency, the architecture utilizes four primary neural components: a contrastive time encoder (G_T), a contrastive frequency encoder (G_F), and two distinct cross-space projectors (R_T and R_F).[9] These projectors, typically structured as multi-layer perceptrons (MLPs), map the respective temporal and spectral representations into a unified, joint time-frequency space.[9]

During pre-training, the architecture optimizes a specifically designed contrastive loss function to force representation alignment:

$$\mathcal{L}_{TF-C} = ||z_T - z_F||^2$$

In this formulation, z_T represents the projected time-domain embedding and z_F represents the projected frequency-domain embedding.[1] To establish robust invariance, the model employs specialized spectral augmentations—such as the strategic addition, removal, or "meiosis" of distinct frequency components—ensuring the model focuses purely on critical affective signatures rather than background noise.[56]

During the fine-tuning phase for ultimate downstream inference, the pipeline concatenates the aligned time-domain and frequency-domain embeddings to form the final sample embedding.[9] This highly contextualized, TF-C aligned token sequence is then submitted to the core Tribe V2-style Transformer, empowering the agentic pipeline with unparalleled accuracy in cognitive state classification and neurological anomaly detection.[9]

5. Implementation Roadmap & Schemas

The successful construction of the Agentic-Wavelet Foundation Pipeline mandates a strict separation of concerns within a highly organized hybrid monorepo environment, governed by rigorous software engineering lifecycle practices.

5.1 Concrete Directory Structure

The system infrastructure is sharply divided between two primary domains: the TypeScript/Bun orchestration agent managing the operational state, and the Python open-source mathematical repository executing the heavy analytical processing.

Directory Path	System Domain	Architectural Purpose

agentic-wavelet-pipeline/	Root	Monorepo root containing global CI/CD pipelines and infrastructure as code.
agent/	Bun/TypeScript	The "Claude Code" inspired orchestration layer.
agent/src/ui/	Bun/TypeScript	React/Ink components rendering the interactive terminal interface.
agent/src/query/	Bun/TypeScript	The QueryEngine handling LLM iteration, streaming, and context compression.
agent/src/coordinator/	Bun/TypeScript	Logic for multi-agent swarm deployments and parallel task execution.
agent/src/mcp/	Bun/TypeScript	Protocol definitions and Zod schemas bridging the TS agent to external tools.
open-source-wst/	Python	The enterprise-grade Wavelet Scattering PyPI package.
open-source-wst/src/eeg_wst/	Python	Core module containing Kymatio integration, PCA algorithms, and data parsing.

5.2 TypeScript Zod Schema for the MCP Tool

The Model Context Protocol requires absolute precision in interface definitions.[36] The following TypeScript Zod schema strictly defines the input parameters for the ExecuteWSTTool, providing the LLM with exact semantic boundaries to prevent execution hallucination.[28]

TypeScript

```typescript
import { z } from "zod";

export const ExecuteWSTSchema = z.object({
  input_directory: z.string().describe("Absolute system path to the directory containing raw EEG files in.edf or.fif format."),
  output_directory: z.string().describe("Absolute system path where the resulting low-dimensional WST NumPy arrays will be persistently saved."),
  sampling_rate: z.number().positive().describe("The uniform sampling frequency of the raw EEG data in Hertz."),
  j_scale: z.number().int().min(1).max(16).describe("The maximum scale (J) of the scattering transform as a power of two. Defines the maximum time-window of the primary low-pass averaging filter."),
  q_wavelets: z.tuple([
    z.number().int().min(1).max(16).describe("Q1: Number of wavelets per octave for the first-order temporal filter bank."),
    z.number().int().min(1).max(16).describe("Q2: Number of wavelets per octave for the second-order temporal filter bank.")
  ]).describe("A precise tuple defining the frequency resolution of the cascaded Morlet wavelets."),
  apply_pca: z.boolean().optional().default(true).describe("Boolean flag to apply Principal Component Analysis for secondary dimensionality reduction post-scattering, crucial for Transformer tokenization."),
  n_components: z.number().int().positive().optional().describe("Number of principal components to retain if PCA dimensionality reduction is activated.")
});

export type ExecuteWSTInput = z.infer<typeof ExecuteWSTSchema>;
```

5.3 Phased Implementation and Edge Inference Strategy

The development lifecycle will proceed through four distinct phases, prioritizing reliability and architectural decoupling. Phase one focuses exclusively on the Python environment, building the eeg_wst package around Kymatio, parametrizing the J and Q values for diverse sampling

rates, and enforcing rigorous unit testing for tensor shape continuity before publishing to PyPI. Phase two scaffolds the Bun/TypeScript CLI, implementing the QueryEngine with its asynchronous generator loop and React/Ink rendering pipeline. Phase three establishes the MCP bridge, instantiating the local MCP server to execute the Python package via JSON-RPC messages validated by the defined Zod schemas. The final phase completes the foundation integration, finalizing the LLM prompts that teach the agent to autonomously evaluate output variances and submit the optimized matrices to the Tribe V2-style Transformer.

To address the severe latency budgets inherent in high-throughput streaming instruments, the architecture incorporates an "Inference at the Edge" strategy.[1] While heavy model training and complex quantum or multi-agent simulations occur offline, the finalized, lightweight mathematical pipelines (the Kymatio WST execution and subsequent tokenization) are distilled and deployed directly to edge nodes.[1] The agent utilizes containerized microservices orchestrated via Kubeflow, ensuring the hybrid Python/TypeScript pipeline remains highly portable and capable of real-time execution.[1] Data Version Control (DVC) and MLflow are utilized throughout the lifecycle to guarantee absolute reproducibility of all datasets and hyperparameters, maintaining strict compliance with enterprise scientific standards.[1]

6. Conclusion

The intersection of autonomous agentic orchestration, highly non-linear signal processing, and biological foundation modeling represents a massive paradigm shift in computational neuroscience. By abandoning the brittle, stationarity assumptions of the Fast Fourier Transform and embracing the robust, deformation-invariant properties of the Wavelet Scattering Transform [1], the proposed architecture guarantees the meticulous preservation of critical, high-frequency neural motifs.

Crucially, embedding this intense mathematical rigor within an autonomous, terminal-native MLOps agent—powered by the Bun runtime, React/Ink UI, and the Model Context Protocol [3]—eradicates the vast manual overhead traditionally associated with biomedical data curation. The enforcement of strict Zod-schema boundaries ensures highly deterministic execution, empowering the QueryEngine to autonomously evaluate mathematical outputs, identify anomalies, and dynamically route data through complex analytical workflows without human supervision.[27]

Finally, by projecting these mathematically optimized, low-dimensional scattering features into a multimodal foundation space utilizing TF-C contrastive learning and TFM-tokenization [1], the pipeline flawlessly bridges the chasm between raw electrical impulses and high-level cognitive mapping, directly echoing the sophisticated achievements of Meta's TRIBE v2.[1] The strategic architectural decision to decouple the WST engine into an open-source PyPI package ensures that this pipeline remains modular, reproducible, and fully equipped to scale alongside the accelerating frontiers of in-silico neuroscience.

Works cited

1. 657045057_4517522795185706_3804826277082824742_n.pdf
2. Time-Frequency Analysis and Wavelets - Sapien Labs | Shaping the Future of Mind Health, accessed on April 3, 2026, https://sapienlabs.org/time-frequency-analysis-and-wavelets/
3. Claude Code Source Leak: Production AI Architecture Patterns from 512000 Lines, accessed on April 3, 2026, https://discuss.huggingface.co/t/claude-code-source-leak-production-ai-architecture-patterns-from-512-000-lines/174846
4. Anthropic Leaked 512,000 Lines of Claude Code Source. Here's What the Code Actually Reveals. : r/ClaudeAI - Reddit, accessed on April 3, 2026, https://www.reddit.com/r/ClaudeAI/comments/1s9ai45/anthropic_leaked_512000_lines_of_claude_code/
5. Zain Hasan | Inside Claude Code: An Architecture Deep Dive, accessed on April 3, 2026, https://zainhas.github.io/blog/2026/inside-claude-code-architecture/
6. What is Model Context Protocol (MCP)? A guide | Google Cloud, accessed on April 3, 2026, https://cloud.google.com/discover/what-is-model-context-protocol
7. User guide — kymatio 0.3.0 documentation, accessed on April 3, 2026, https://www.kymat.io/userguide.html
8. An interpretable alternative to convolutional neural networks: the scattering transform | JOV, accessed on April 3, 2026, https://jov.arvojournals.org/article.aspx?articleid=2784695
9. Self-Supervised Contrastive Pre-Training For Time Series - Zitnik Lab, accessed on April 3, 2026, https://zitniklab.hms.harvard.edu/projects/TF-C/
10. (PDF) Single-Channel EEG Tokenization Through Time-Frequency Modeling, accessed on April 3, 2026, https://www.researchgate.net/publication/389315838_Single-Channel_EEG_Tokenization_Through_Time-Frequency_Modeling
11. Tokenizing Single-Channel EEG with Time-Frequency Motif Learning - arXiv, accessed on April 3, 2026, https://arxiv.org/html/2502.16060v4
12. Tokenizing Single-Channel EEG with Time-Frequency Motif Learning [Quick Review] - Liner, accessed on April 3, 2026, https://liner.com/review/tokenizing-singlechannel-eeg-with-timefrequency-motif-learning
13. What is MLOps? An Overview of Machine Learning Operations - eWeek, accessed on April 3, 2026, https://www.eweek.com/artificial-intelligence/mlops-machine-learning-operations/
14. Claude Code Agent — Complete Architecture Deep Dive - GitHub Gist, accessed on April 3, 2026, https://gist.github.com/yanchuk/0c47dd351c2805236e44ec3935e9095d
15. weimeng23/claude-code-src: First version of the forked ... - GitHub, accessed on April 3, 2026, https://github.com/weimeng23/claude-code-src
16. Claude Code Architecture Explained: Agent Loop, Tool System, and Permission

Model (Rust Rewrite Analysis) - DEV Community, accessed on April 3, 2026, https://dev.to/brooks_wilson_36fbefbbae4/claude-code-architecture-explained-agent-loop-tool-system-and-permission-model-rust-rewrite-41b2
17. INK: The React Library POWERING AI Terminal Tools (For Now...) - YouTube, accessed on April 3, 2026, https://www.youtube.com/watch?v=qSZwx5_EmSA
18. claude-code/docs/architecture.md at main - GitHub, accessed on April 3, 2026, https://github.com/codeaashu/claude-code/blob/main/docs/architecture.md
19. Claude Code's Source Code Exposed — Every System Explained From Scratch (512K Lines) - DEV Community, accessed on April 3, 2026, https://dev.to/ishaaan/claude-codes-source-code-exposed-every-system-explained-from-scratch-512k-lines-4blp
20. Deconstructing the Claude Code Leak: Inside Anthropic's Agentic AI "Secret Sauce" | by Jiten Oswal | Apr, 2026 | Medium, accessed on April 3, 2026, https://medium.com/@jiten.p.oswal/deconstructing-the-claude-code-leak-inside-anthropics-agentic-ai-secret-sauce-e17f594338f3
21. Claude Code's source just leaked — I extracted its multi-agent orchestration system into an open-source framework that works with any LLM - Reddit, accessed on April 3, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1s8xj2e/claude_codes_source_just_leaked_i_extracted_its/
22. The Great Claude Code Leak of 2026: Accident, Incompetence, or the Best PR Stunt in AI History? - DEV Community, accessed on April 3, 2026, https://dev.to/varshithvhegde/the-great-claude-code-leak-of-2026-accident-incompetence-or-the-best-pr-stunt-in-ai-history-3igm
23. model-context-protocol-resources/guides/mcp-server-development-guide.md at main - GitHub, accessed on April 3, 2026, https://github.com/cyanheads/model-context-protocol-resources/blob/main/guides/mcp-server-development-guide.md
24. Beyond APIs: Lessons from Building with the Model Context Protocol (MCP) - Medium, accessed on April 3, 2026, https://medium.com/@wim.henderickx/beyond-apis-lessons-from-building-with-the-model-context-protocol-mcp-dc85ca83bb89
25. What is the Model Context Protocol (MCP)? - Databricks, accessed on April 3, 2026, https://www.databricks.com/blog/what-is-model-context-protocol
26. The Model Context Protocol (MCP): A Comprehensive Technical Report - DEV Community, accessed on April 3, 2026, https://dev.to/luzkalidgm/the-model-context-protocol-mcp-a-comprehensive-technical-report-470d
27. MCPmed: a call for Model Context Protocol-enabled bioinformatics web services for LLM-driven discovery - PMC, accessed on April 3, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC12927880/
28. Model Context Protocol (MCP): Build Your Own in 5 Minutes | Upstash Blog, accessed on April 3, 2026, https://upstash.com/blog/build-your-own-mcp
29. Zod Schema Validation Skill for Claude Code | AI Tools - MCP Market, accessed on April 3, 2026, https://mcpmarket.com/tools/skills/zod-schema-form-validation

30. CASCADE: Cumulative Agentic Skill Creation through Autonomous Development and Evolution - arXiv, accessed on April 3, 2026, https://arxiv.org/html/2512.23880v2
31. Get structured output from agents - Claude API Docs, accessed on April 3, 2026, https://platform.claude.com/docs/en/agent-sdk/structured-outputs
32. Haystack vs LlamaIndex: Which One's Better at Building Agentic AI Workflows - ZenML Blog, accessed on April 3, 2026, https://www.zenml.io/blog/haystack-vs-llamaindex
33. Programmatic tool calling - Claude API Docs, accessed on April 3, 2026, https://platform.claude.com/docs/en/agents-and-tools/tool-use/programmatic-tool-calling
34. What Claude Code's Source Leak Actually Reveals | by Marc Bara | Mar, 2026 | Medium, accessed on April 3, 2026, https://medium.com/@marc.bara.iniesta/what-claude-codes-source-leak-actually-reveals-e571188ecb81
35. In the wake of Claude Code's source code leak, 5 actions enterprise security leaders should take now | VentureBeat, accessed on April 3, 2026, https://venturebeat.com/security/claude-code-512000-line-source-leak-attack-paths-audit-security-leaders
36. Level Up Your AI Skills, Build a Custom MCP from Scratch | Alberto Corrales Garcia, accessed on April 3, 2026, https://albertocorrales.com/posts/Build-your-own-MCP/
37. Tools - Model Context Protocol, accessed on April 3, 2026, https://modelcontextprotocol.io/specification/draft/server/tools
38. Zod v4 Schema Expert | Claude Code Skill for Validation - MCP Market, accessed on April 3, 2026, https://mcpmarket.com/tools/skills/zod-v4-schema-expert
39. Python Package Structure & Code - pyOpenSci, accessed on April 3, 2026, https://www.pyopensci.org/python-package-guide/package-structure-code/intro.html
40. Wavelets-based Feature Extraction - Part2: Wavelet Scattering Transform - YouTube, accessed on April 3, 2026, https://www.youtube.com/watch?v=JyonDStplsY
41. Classification of alcoholic EEG signals using wavelet scattering transform-based features - New Zealand Brain Research Institute, accessed on April 3, 2026, https://www.nzbri.org/resources/publications/658/Buriro_Computers-in-Biology-and-Medicine_2021.pdf
42. arXiv:2304.14919v1 [eess.SP] 26 Apr 2023, accessed on April 3, 2026, https://arxiv.org/pdf/2304.14919
43. Efficient EEG Feature Learning Model Combining Random Convolutional Kernel with Wavelet Scattering for Seizure Detection - World Scientific Publishing, accessed on April 3, 2026, https://www.worldscientific.com/doi/10.1142/S0129065724500606
44. A Wavelet Scattering Feature Extraction Approach for Deep Neural Network Based Indoor Fingerprinting Localization - PMC, accessed on April 3, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC6514606/

45. Plot the 1D wavelet filters — kymatio 0.3.0 documentation, accessed on April 3, 2026, https://www.kymat.io/gallery_1d/plot_filters.html
46. Kymatio: Scattering Transforms in Python | Georgios Exarchakis, accessed on April 3, 2026, https://exarchakis.net/publication/kymatio-scattering-transforms-in-python/
47. Kymatio: Wavelet scattering in Python - v0.3.0 “Erdre” — kymatio 0.3.0 documentation, accessed on April 3, 2026, https://www.kymat.io/
48. Kymatio: Scattering Transforms in Python - Journal of Machine Learning Research, accessed on April 3, 2026, https://www.jmlr.org/papers/volume21/19-047/19-047.pdf
49. Classification using Scattering Transform - Kaggle, accessed on April 3, 2026, https://www.kaggle.com/code/alermd/classification-using-scattering-transform
50. Decoding Visual Art Preferences From EEG Signals Using Wavelet Scattering Transform | Request PDF - ResearchGate, accessed on April 3, 2026, https://www.researchgate.net/publication/393742390_Decoding_Visual_Art_Preferences_From_EEG_Signals_Using_Wavelet_Scattering_Transform
51. Hierarchical structure of wavelet scattering transform process - ResearchGate, accessed on April 3, 2026, https://www.researchgate.net/figure/Hierarchical-structure-of-wavelet-scattering-transform-process_fig6_364333962
52. Simple packaging - Scientific Python Development Guide, accessed on April 3, 2026, https://learn.scientific-python.org/development/guides/packaging-simple/
53. The Most Complete Guide for Creating a Good PyPI Package | by Elise Landman - Medium, accessed on April 3, 2026, https://medium.com/data-science/the-most-complete-guide-for-creating-a-good-pypi-package-acb5420a03f8
54. How to Publish an Open-Source Python Package to PyPI, accessed on April 3, 2026, https://realpython.com/pypi-publish-python-package/
55. Python Packages: Installation & Management Best Practices - Anaconda, accessed on April 3, 2026, https://www.anaconda.com/guides/python-packages
56. EEG2Rep: Enhancing Self-supervised EEG Representation Through Informative Masked Inputs - arXiv, accessed on April 3, 2026, https://arxiv.org/html/2402.17772v1
57. TRIBE v2 - atmeta.com, accessed on April 3, 2026, https://aidemos.atmeta.com/tribev2
58. Meta Releases TRIBE v2: A Brain Encoding Model That Predicts fMRI Responses Across Video, Audio, and Text Stimuli - MarkTechPost, accessed on April 3, 2026, https://www.marktechpost.com/2026/03/26/meta-releases-tribe-v2-a-brain-encoding-model-that-predicts-fmri-responses-across-video-audio-and-text-stimuli/
59. GitHub - mims-harvard/TFC-pretraining: Self-supervised contrastive learning for time series via time-frequency consistency, accessed on April 3, 2026, https://github.com/mims-harvard/tfc-pretraining
60. Self-Supervised Contrastive Pre-Training For Time Series via Time-Frequency Consistency | OpenReview, accessed on April 3, 2026,

https://openreview.net/forum?id=OJ4mMfGKLN
61. Self-Supervised Contrastive Pre-Training for Time Series via Time-Frequency Consistency - OpenReview, accessed on April 3, 2026, https://openreview.net/pdf?id=OJ4mMfGKLN
62. Time-Frequency Consistency (TF-C), the first realization of prior learning in time series with self-supervised contrasted learning | AI-SCHOLAR, accessed on April 3, 2026, https://ai-scholar.tech/en/articles/time-series/TFC-pretraining
63. [2206.08496] Self-Supervised Contrastive Pre-Training For Time Series via Time-Frequency Consistency - arXiv, accessed on April 3, 2026, https://arxiv.org/abs/2206.08496